

Persense Financial Math Explainer

A consolidated, plain-language account of the financial mathematics behind Per%Sense — built up from one idea to the complete approach, drawing on both the original application and its modern re-derivation, and explaining why we can trust it.

What this is. A conceptual explainer of the *mathematics* Per%Sense uses to value loans, mortgages, cash-flow streams, and life-contingent payments. It builds the ideas in layers and stays high-level: no program code, no dependence on how either implementation is written.

Source. This document is a **synthesis of two independent realizations of the same mathematics**: the original DOS application, and a faithful modern re-derivation (the Go port). Where they agree, that agreement is evidence the mathematics is right; where the modern effort had to reconstruct or clarify something, this document says so plainly.

Who it is for. Anyone who wants to understand *what* Per%Sense computes, *why* the answers are correct, and *how confident* we can be in them. No prior finance background is assumed.

Contents

- 1 Introduction: one mathematics, two realizations**
- 2 The time value of money**
- 3 Measuring time: day-count conventions**
- 4 The annuity: a geometric series at the core**
- 5 Amortizing a loan**
- 6 The present value of arbitrary cash flows**
- 7 Mortgages and the true cost of borrowing**
- 8 Life contingencies — and a reconstruction**
- 9 How a calculation actually proceeds**
- 10 The iterative solver in depth**
 - 10.1 The error function · 10.2 Newton & the secant · 10.3 Safeguards · 10.4 Bisection · 10.5 Two unknowns · 10.6 Round-trip proof
- 11 The subtle frontiers**
- 12 How we know it is right**
- 13 Why it all works**

1 Introduction: one mathematics, two realizations

Per%Sense presents as several calculators — a mortgage sizer, a loan amortizer, a present-value worksheet, an actuarial annuity tool. Beneath them is a single mathematical idea, applied with growing richness; and that idea now exists in two independent implementations whose agreement is part of why we trust it.

The idea is the **time value of money**: a dollar promised later is worth less than a dollar today, by an amount governed by an interest rate. Every Per%Sense screen (a) records a set of dated money movements, (b) translates them to a common point in time so they can be compared and added, and (c) solves for whatever the user left blank. Two themes recur:

- **Everything reduces to discounting and summing.** A single balloon, a level mortgage payment, an escalating pension, a benefit paid only on death — all are handled by moving each amount through time with an interest factor and adding the pieces. Where payments are regular, the sum collapses into one compact geometric-series formula, the engine the whole program turns on.
- **The user fills in the blanks.** Per%Sense treats each screen as a small system of equations in which any single variable can be the unknown. Sometimes the algebra isolates it directly; sometimes the unknown is buried inside the equation and must be found by successive approximation.

What makes this document a *synthesis* is its second vantage point. The same financial mathematics was realized twice — once in the original DOS program, and again in a careful modern port built to reproduce it. Re-deriving the mathematics from scratch did three useful things: it **confirmed** the relationships (two independent implementations that agree to the cent are unlikely to share an error), it **pinned down** the subtle edge cases where a clean formula degenerates, and in one area — the actuarial engine, whose original source was lost — it required **reconstructing** the mathematics from the surrounding evidence. We will weave that perspective through the exposition and gather it explicitly in Sections 10 and 11.

The shape of this document. We start with the single building block (the time value of money), add the geometric series that lets us value whole streams at once, then apply that engine to loans, general present-value problems, mortgages, and life-contingent payments. We then explain how the program inverts these relationships, catalog the subtle frontiers the two-implementation effort isolated, and finally lay out the basis for confidence in the results.

2 The time value of money

Money has a price for time. With a positive interest rate, a dollar now beats a promised future dollar, because today's dollar can be put to work meanwhile. Asking “what is a future dollar worth today?” is **discounting** — the operation Per%Sense performs most.

Per%Sense works internally with **continuous compounding**: money grows over t years at rate r by the factor $e^{r \cdot t}$, so a payment of C due t years out is worth

$$\text{Present value} = C \cdot e^{-r \cdot t}.$$

Both implementations choose this convention for the same reason: the algebra is clean. Adjacent periods' factors simply multiply, fractional years need no special cases, and payment streams telescope neatly. The continuously-compounded rate is the *force of interest* — the program's internal “true rate.”

Two views of one rate. People quote rates as, say, “6% compounded monthly,” not as a continuous force. So the program keeps both views and converts at the boundary: a nominal rate compounded n times a year, y , and the internal continuous rate, r , are exact inverses, $y = n(e^{r/n} - 1)$ and $r = n \cdot \ln(1 + y/n)$. The user enters and reads a yield; the engine computes in the continuous rate. Both implementations keep this boundary in exactly the same place, which is part of why their internal numbers line up.

WORKED EXAMPLE — DISCOUNTING, AND THE TWO VIEWS OF A RATE

A promise of **\$10,000 in 5 years**, at a 6% continuous rate, is worth today

$$\$10,000 \times e^{-0.06 \times 5} = \$10,000 \times e^{-0.30} = \$10,000 \times 0.7408 = \$7,408.$$

And a rate quoted as **6% compounded monthly** becomes the internal continuous rate

$$r = 12 \times \ln(1 + 0.06/12) = 12 \times \ln(1.005) = 5.985\%,$$

which converts straight back: $12 \times (e^{0.05985/12} - 1) = 6.00\%$. Both implementations keep this conversion at the same boundary — the continuous rate inside, the quoted rate at the edges.

3 Measuring time: day-count conventions

Interest is rate times *time* times principal, so we must answer: how much of a year lies between two dates? There is no universal answer — it is a market convention, and the wrong one gives a subtly wrong number. Per%Sense supports the common conventions:

Convention	How a year is measured	Typical use
30/360	Every month is 30 days, every year 360, with small month-end corrections.	Many loans and bonds; clean payment amounts.
Actual / 365–366	Real calendar days divided by the actual year length (366 in leap years).	Loans accruing on actual days outstanding.
Actual / 365.25	Real days over an average year length.	The present-value valuation screens.

The differences look small but compound across a long schedule — a thirty-year loan has 360 interest calculations, and a fraction-of-a-day error in each moves the final balance by real money. Both implementations reproduce these conventions identically, down to the month-end corrections; getting the day count to agree was, in fact, one of the more demanding parts of confirming the two engines match. Dates are handled as serial day-numbers so that the gap between dates is a subtraction and advancing by a month or quarter behaves sensibly at month-ends.

WORKED EXAMPLE — THE SAME SIX MONTHS, TWO CONVENTIONS

From 1 January to 1 July 2024 is exactly half a calendar year — but “half” depends on the rule. Under **30/360** the span is 6 months ÷ 12 = **0.5000 year**. Under an **actual-day** rule it is 182 real days ÷ 366 (a leap year) = **0.4973 year**. On \$100,000 at 8%, one period's interest is therefore **\$4,000.00** the first way and **\$3,978.14** the second — a \$22 gap from the choice of convention alone. Getting both engines to agree on figures like these, down to the month-end corrections, was one of the more demanding parts of confirming they match.

4 The annuity: a geometric series at the core

Discounting one payment is easy. The reason Per%Sense values a thirty-year stream instantly — in either implementation — is that a *regular* stream collapses into one formula.

Take a level stream: the same payment A at the end of each period, for n periods. If one period's discount factor is v (just under 1), the payments are worth Av , Av^2 , ..., Av^n , and the total is a **geometric series**:

$$PV = A \cdot (v + v^2 + \dots + v^n) = A \cdot v \cdot (1 - v^n) / (1 - v).$$

This **annuity factor** is the most important formula in Per%Sense. It replaces a sum over hundreds of payments with one expression in the discount factor v and the count n . A loan payment is the number that makes this value equal the principal; a mortgage relationship is an annuity relationship; a pension's worth is an annuity value; an escalating stream is the same with a modified ratio; a life annuity is the same with each term probability-weighted. Both implementations are built on this one spine.

The degenerate case — and why the two implementations agree on it. When the rate is exactly zero, $v = 1$ and the formula reads $0/0$; the correct value is plainly $n \cdot A$ (with no interest, each payment is worth its face). Both implementations detect when the rate is close enough to zero that the general formula would misbehave and substitute the plain count. Confirming that *both* draw the cutoff the same way — so their answers agree right up to the boundary — was one of many small checks that together establish the engines compute the same mathematics.

WORKED EXAMPLE — VALUING A FIVE-YEAR STREAM

The present value of **\$1,000 a year for 5 years at 8%** uses the annuity factor with one-year discount factor $v = 1/1.08 = 0.9259$:

factor = $v(1 - v^5)/(1 - v) = 3.9927$, so **PV** = $\$1,000 \times 3.9927 = \$3,992.71$.

Five separate discountings replaced by one expression. At a **0% rate** the factor would instead be the plain count, 5, giving exactly **\$5,000** — the degenerate case both implementations handle the same deliberate way.

5 Amortizing a loan

An amortizing loan is repaid by regular payments, each part interest and part principal, until the balance reaches zero. Per%Sense both builds the schedule forward and solves for any missing piece.

The forward schedule is one recurrence iterated. At the start of a period the borrower owes B ; interest accrues on it; the payment covers the interest first and the rest reduces principal:

$$B_{\text{new}} = B + \text{interest} - \text{payment}, \quad \text{interest} = B \cdot (\text{rate for one period}).$$

Iterate and the whole schedule appears: early payments are mostly interest, later ones mostly principal. **The level payment** that retires the loan in n periods is the annuity inversion of Section 4 — given principal, rate, and count, solve the annuity formula for the payment. Because the principal enters linearly, this is exact algebra, no searching.

Real loans depart from the textbook case in specific ways, and Per%Sense treats each as a precise modification of the recurrence rather than an ad-hoc tweak: **payment in advance** (payments at the period's start, worth slightly more, so a touch smaller); an **odd first period** (the stub from closing to the first payment is prorated by its true length); the **Rule of 78** (interest front-loaded by sum-of-the-digits weights); and the **U.S. Rule** (unpaid interest tracked separately and never itself charged interest). Beyond level payments, the cash-flow stream can be sculpted with **balloons, prepayments, rate adjustments, a moratorium, a target, and skipped months** — none of which change the engine: the period-by-period walk simply consults, at each date, whatever events apply.

Where the second implementation earned its keep. These variations interact in combinations the original handled but never spelled out — an odd first period *and* payment-in-advance *and* a balloon, for instance. Re-deriving the engine forced each combination to be made explicit and checked against the original to the cent. Several of these “frontiers” are catalogued in Section 10; the point here is that the amortization mathematics is now understood at a finer grain than either implementation alone documents.

WORKED EXAMPLE — A CAR LOAN

Borrow **\$25,000 over 5 years (60 monthly payments) at 6%** (0.5% per month). The level payment is the annuity inversion — the payment whose 60 discounted instalments total \$25,000:

$$\text{payment} = \$25,000 \times 0.005 / (1 - 1.005^{-60}) = \$483.32.$$

The first payment splits into **\$125.00 interest** ($\$25,000 \times 0.5\%$) and **\$358.32 principal**, leaving \$24,641.68. Iterating that split, the balance reaches exactly zero at payment 60 — a result both engines reproduce row for row.

6 The present value of arbitrary cash flows

The present-value screen generalizes the loan: value *any* collection of dated amounts against a chosen valuation date.

Two kinds of flow are allowed: **single payments** (a lump on a date) and **periodic streams** (a level amount, so many times a year, between two dates). Each lump is discounted by the time-value factor; each stream by the annuity factor; the grand total is their sum as of the common “as-of” date. Same discount-and-sum recipe, over freely chosen flows.

Escalating payments. A stream may carry a cost-of-living rate, so each payment grows. Now two forces act on each: growth lifts it, discounting shrinks it. Their net effect per period is a single combined ratio — escalation against discount — and the stream is again a geometric series with that ratio in place of the plain discount factor. The annuity machinery carries straight over.

When growth equals the discount rate. If a stream grows at exactly the discount rate, the two forces cancel: every payment, escalated then discounted, is worth the same today, the ratio becomes 1, and the value is just the count times one payment's present value — the same degeneracy as the zero-rate annuity. Both implementations handle it the same deliberate way. This particular case is a useful litmus test: a naive implementation divides by zero here, so the fact that *both* Per%Sense engines return the clean limiting value is a small but telling sign that the mathematics was understood, not merely transcribed.

Perpetuities. A stream with no end has a finite value only if discounting outruns growth; otherwise the value is infinite, and Per%Sense says so rather than returning a meaningless figure. That is the honest mathematical answer, and both implementations give it.

WORKED EXAMPLE — AN ESCALATING PENSION

A pension pays **\$12,000 a year, rising 3% annually, for 20 years**, discounted at 6%. Growth and discounting combine into a single net ratio per year, and the geometric sum gives a present value of about **\$174,800**. Now suppose the pension grew at the full 6% — matching the discount rate. Then every payment, escalated and then discounted, is worth exactly the first payment's value, \$11,321, so the whole stream is simply **$20 \times \$11,321 = \$226,415$** — the growth-equals-discount degeneracy, where the geometric ratio becomes 1. This is a useful litmus test: a naive engine divides by zero here, so the fact that both Per%Sense engines return the clean limit is a small sign the mathematics was understood, not merely transcribed.

7 Mortgages and the true cost of borrowing

A mortgage screen is an annuity problem in the vocabulary of home buying, plus two questions about comparing loans: what does it really cost, and which of two loans is cheaper?

The web of relationships. The amount financed is price less down payment; the cash needed is down payment plus points on the loan; the monthly payment is the annuity payment that amortizes the financed amount over the term, plus a flat tax-and-insurance allowance that is added but never amortized. Because the relationships are mutually consistent, the user specifies whichever subset they know and the program rearranges the same relationships to fill in the rest. A balloon is one extra future lump folded into the same equation.

APR as an internal rate of return. The quoted rate is not the true cost, because the borrower pays points up front and so receives less than the face amount. The annual percentage rate is the single rate at which

$$(\text{present value of all payments and any balloon}) = (\text{amount financed}) \times (1 - \text{points}).$$

The right side is the net cash actually advanced; the left discounts everything the lender gets back. The APR is the rate equating them — an internal rate of return — always at least the note rate, more so over a shorter loan. It cannot be written in closed form and is found by the search methods of Section 9.

The crossover. Comparing a low-rate/high-points loan with a high-rate/low-points one, Per%Sense finds the holding period at which their effective costs are exactly equal — below it one loan wins, above it the other. This is two cost curves intersecting: a pair of equations solved simultaneously for the crossover rate and time. A balloon puts a step in a loan's cost curve, so the crossover may sit exactly on a balloon date — which the program checks directly when the smooth search stalls.

WORKED EXAMPLE — SIZING A MORTGAGE AND ITS TRUE COST

On a **\$200,000 home, 20% down, 2 points, 30 years at 6%**: the amount financed is \$160,000; the cash needed is the \$40,000 down payment plus \$3,200 of points = **\$43,200**; and the monthly principal-and-interest payment is the annuity payment for \$160,000 at 0.5% per month over 360 months = **\$959.28**. Because the 2 points mean the borrower effectively receives only \$156,800, the true cost — the rate at which those 360 payments discount back to \$156,800 — works out to an **APR of about 6.24%**, a little above the 6% note rate. That last figure has no closed form; it is found by the iterative solver of Section 10.

8 Life contingencies — and a reconstruction

The most sophisticated screen values payments that happen only under a condition involving human life. The mathematics extends everything so far — and it is the one place where the modern effort had to rebuild rather than reproduce.

The mathematics

A payment made only if a certain person is alive on its date is, from today, uncertain. The right value of an uncertain payment is its **expected** present value — the ordinary present value times the *probability* the payment happens:

$$\text{value} = (\text{amount}) \times (\text{discount factor}) \times (\text{probability the condition holds}).$$

Those probabilities come from a **mortality table** of how many survive to each age. The chance a person aged x survives t more years is the ratio of survivors at age $x+t$ to survivors at age x , interpolated smoothly between whole-year entries. Each life-contingent payment is weighted by its survival probability, making the annuity a **life annuity** — valued payment by payment, since the weight differs at every date. With two independent lives, conditions combine by elementary probability: both-alive is the product of survival probabilities; either-alive is one minus the product of death probabilities; exactly-one-alive multiplies one survival by the other's death. A **payment on death** is the sum, over each possible year of death, of the benefit times the probability of dying that year times the discount factor — an expectation over *when* death occurs. In every case the principle is unchanged: discount each possible cash flow, weight it by the probability it occurs, and add.

The reconstruction

This engine was rebuilt, not ported. The original program's actuarial computational core — the routines that read the mortality table and produced survival and death-benefit values — was missing from the surviving materials. What remained was the *integration*: the surrounding code showed exactly how those values were *used* (each cash flow multiplied by a survival probability; a death-benefit value added to the total; the symmetry of adding these on the way in and removing them on the way out). From that recovered contract, plus standard actuarial mathematics, the modern effort reconstructed the missing core.

This makes the life-contingency mathematics **solid in structure but partly unverifiable in detail**, and intellectual honesty requires marking the line. What is firmly established: that each cash flow is weighted by a survival probability; how the seven one- and two-life conditions combine; that the death benefit's value is an expectation over the time of death; that a date cannot be solved from a life-contingent value (the weight itself depends on that date); and that the survival weight is applied separately at every payment date. What cannot be confirmed against the original, because the source is gone: the exact *numbers* in the mortality table, the precise convention for interpolating between ages, and the exact way the death-benefit expectation is discretized. The reconstruction uses standard, defensible choices for these, and is correct against canonical actuarial mathematics — but it is the one part of Per%Sense whose fidelity to the original cannot be checked to the cent. Section 11 returns to what this means for confidence.

WORKED EXAMPLE — A LIFE ANNUITY (ILLUSTRATIVE SURVIVAL)

Value **\$10,000 a year while a 70-year-old survives**, discounted at 5%. Each year's payment is worth its discounted value times the chance of being alive to collect it. Using illustrative survival probabilities:

Year	Discount factor	Survival probability	Expected value
1	0.9512	0.97	\$9,227
2	0.9048	0.93	\$8,415
3	0.8607	0.89	\$7,660
4	0.8187	0.84	\$6,877
5	0.7788	0.79	\$6,153

Over these first five years the life annuity is worth **\$38,332**, against **\$43,142** for a certain annuity that pays regardless — the roughly \$4,800 difference is the mortality discount. The structure of this calculation is firmly recovered; the survival numbers, as Section 8 explains, are the part that had to be reconstructed and cannot be checked against the original. (The figures here are illustrative; the principle is exact.)

9 How a calculation actually proceeds

We have the ingredients. Now follow a single calculation from “compute” to answer, so the mechanics are concrete — and so we can be precise about what the two implementations do identically.

Step one: read the screen by counting

Every row has a few defining quantities — a single payment is defined by date, amount, and present value; a periodic stream by two dates, an amount, and a value; the screen as a whole by a discount rate and an as-of date. Per%Sense begins by *counting*, on each row, how many are supplied and how many are blank, and the count classifies the row:

What the count shows	Meaning	Action
Exactly the right number present	Fully specified	Compute it <i>forward</i> .
Missing exactly one	Contains an unknown	Solve it <i>backward</i> .
Too few present	Under-determined	Ask for more input.
More than enough, conflicting	Over-determined	Warn; do not silently choose.

This counting is the mathematical content of “fill in the blanks”: one unknown is one equation in one unknown (solvable); zero unknowns is something to evaluate; two or more is ambiguous. Both implementations make exactly this classification, by the same counting — which is why, given the same screen, they always agree on *what* is being asked before they agree on the answer.

Step two: choose a direction

From the per-row classification the program sets the screen's direction. Rate and as-of date known and every active row fully specified → run **forward** (discount and sum). Exactly one quantity blank and a target to hit → run **backward** (solve that quantity). Two quantities blank → no unique answer, and the program says so.

Step three: invert directly, or set up a search

Once the single unknown is identified, the program asks whether it can be isolated algebraically. **Linear** unknowns — a loan amount, a level payment, a single lump's value — are rearranged and computed in one exact step. **Tangled** unknowns — a rate inside the discount exponentials, a date that changes both the count of payments and their discounting, a term that changes the count — cannot be freed by rearrangement, so the program builds an *error function* and hands it to the iterative solver of Section 10.

A worked pair of calculations

Forward: a \$10,000 loan at 1% per month for 12 payments. The level payment is the annuity inversion — the payment whose discounted twelve-fold stream equals \$10,000 — about **\$888.49**. The schedule is then the recurrence iterated: month 1 charges \$100.00 interest and \$788.49 principal, leaving \$9,211.51; month 2 charges \$92.12 and \$796.37, leaving \$8,415.14; and so on until the twelfth payment lands the balance exactly on zero. Backward: supply the \$888.49 payment and \$10,000 amount but leave the *rate* blank. The rate is tangled, so the program defines a number measuring how wrong a guessed rate is and drives it to zero — the solver, which both implementations carry out by the same steps.

10 The iterative solver in depth

When the unknown cannot be isolated, Per%Sense finds it by successive approximation. The method is the same across every screen — and, notably, the modern re-derivation reproduces not just the original's answers but its exact search behavior.

10.1 The error function: turning a question into a root

The solver never works on the unknown directly. It works on an **error function** — one number, depending on a guess x , positive when the guess is too far one way, negative the other way, zero when it is right. Call it $g(x)$. Solving the screen becomes the universal problem of **finding the root** of g : the value x^* where $g(x^*) = 0$. Per%Sense uses two natural error functions:

- **Leftover-balance error.** For a loan with an unknown payment, rate, or amount, plug in the guess, run the whole amortization schedule, and read the balance left at the end. A correct guess retires the loan exactly (zero left); too small a payment leaves a positive balance, too large a negative one.
- **Value-gap error.** For a rate, date, or term, compute the present value the guess produces and subtract the target it must hit: $(\text{value at guess}) - (\text{target})$, zero exactly when the guess reproduces the target.

Both are chosen to move steadily in one direction as the guess changes — a higher rate lowers a present value, a larger payment lowers a leftover balance — guaranteeing a single crossing and an unambiguous root.

10.2 Newton's method and the secant variant

Newton's method treats g , near the current guess, as a straight line — its tangent — and jumps to where that line crosses zero:

$$x_{\text{next}} = x - g(x) / g'(x).$$

When g is smooth, this roughly *doubles* the number of correct digits each step — a handful of iterations to the cent. But Per%Sense's errors have no tidy slope formula (“run the schedule and read the leftover” is a simulation, not a differentiable expression), so the program estimates the slope from the two most recent evaluations — the **secant** method, Newton with a chord:

$$x_{\text{next}} = x_{\text{now}} - g(x_{\text{now}}) \cdot (x_{\text{now}} - x_{\text{prev}}) / (g(x_{\text{now}}) - g(x_{\text{prev}})).$$

It needs no derivative, treats the error as a black box, and converges almost as fast. A worked illustration with the \$10,000 / \$888.49 / 12-payment loan: take the error to be the annuity value at a guessed monthly rate minus \$10,000. At a guess of 0.8% the value is about \$128 too high (rate too low); at 1.2% about \$126 too low. The chord through those points crosses zero at about **1.00%** — the loan's rate, in one secant step. Both engines run this identical iteration.

That single step recovered a rate we already knew. The real power shows when the answer is genuinely unknown and the search must take several steps — so let us solve, in full, for a quantity with no closed form: the true cost of a loan with points. Both engines trace the identical sequence below.

WORKED EXAMPLE — SOLVING FOR AN APR, STEP BY STEP

Take the loan of Section 9 — payment \$888.49 a month for 12 months — but now charge **2 points**, so the borrower actually receives $\$10,000 \times (1 - 0.02) = \$9,800$. The true monthly rate is the one at which the twelve payments discount back to \$9,800. The error function is

$$g(r) = (\text{value of the payments at rate } r) - \$9,800,$$

which we drive to zero. Starting from two nearby guesses, the secant iteration runs:

Step	Guess (monthly rate)	Value of the payments	Error g
0 (start)	1.000%	\$10,000.0	+\$200.0
1 (start)	1.200%	\$9,873.8	+\$73.8
2 (secant)	1.317%	\$9,802.9	+\$2.9
3 (secant)	1.322%	\$9,799.9	-\$0.1

Each new guess is the point where the straight line through the two previous (rate, error) points crosses zero. The error collapses $+\$200 \rightarrow +\$74 \rightarrow +\$3 \rightarrow -\0.1 — settling on a monthly rate of about **1.322%**, an **APR of roughly 15.9%**, well above the 12% note rate because two points on a one-year loan are costly. Four evaluations reach the cent.

Why this converges — and why it must. Two properties of the error function guarantee success. First, it is **monotonic**: raising the rate lowers the value of a fixed set of future payments, so g decreases steadily as the rate rises and crosses zero exactly once — there is a single answer, and no false one to be trapped by. Second, it is **gently curved**: over a small interval the value-versus-rate relationship is very nearly a straight line, so the chord the secant draws through two recent points is an excellent stand-in for the true curve, and the place where that chord hits zero lands close to the real root. Each step then uses two fresher, closer points, so the straight-line stand-in grows ever more accurate and the remaining error shrinks faster and faster — visible above as the error falls by a factor of three, then twenty-five, then thirty. Because both engines run this same iteration with the same safeguards, they descend this very sequence of guesses in lock-step to the same final cent.

10.3 Safeguards: making the search robust

A bare secant can misbehave, so the iteration is wrapped in safeguards: a **smart first guess** (an APR seeded with the note rate plus amortized points; a rate with the payment-to-principal ratio), **step damping** (each jump capped so a bad slope estimate cannot fling the guess to an absurd value), **best-so-far** (remember the guess with the smallest error seen and be ready to return it), **restart** (re-seed from a different point if the search stalls), and a **divergence brake** (accelerate toward termination if steps stop shrinking). The iteration stops on whichever comes first: the step falls below a tolerance, the error itself falls within an acceptable band (a balance within half a cent, a date within a day), or a maximum iteration count — the last a safety valve, after which non-convergence is reported honestly rather than a number from a search that never settled.

The two engines agree on the path, not just the destination. Because the safeguards — the specific damping, the best-so-far rule, the restart and divergence logic — can subtly affect the final cent, the modern re-derivation reproduces them deliberately, down to fine points of ordering. The payoff is that the two implementations converge along the *same* sequence of guesses to the *same* answer, so their agreement tests the solver itself, not merely the formulas it is solving.

10.4 When the fast method fails: bisection

Newton and the secant assume a smooth error. Some of Per%Sense's are not: the amortization schedule “forces” the final payment to clear the balance and stops, so as a guess changes the number of payments can flip and the leftover-balance error **jumps** — a step function. A tangent across a jump points nowhere useful. For these the program uses **bisection**: bracket the root between a guess that is too low and one too high (opposite error signs), then repeatedly take the midpoint and keep whichever half still straddles zero. Each step *halves* the uncertainty; bisection cannot be fooled by a discontinuity and cannot diverge. The cost is speed — a fixed gain per step instead of doubling — so it is reserved for exactly the rough cases. Choosing the fast method where the error is smooth and bisection where it is not is the solver's central judgement, and both implementations draw that line in the same place.

10.5 Two unknowns at once: the crossover

The mortgage crossover solves for two unknowns — a rate and a holding time — satisfying two equations at once. Newton generalizes: estimate how each error responds to a small nudge in each unknown (a two-by-two table of sensitivities), solve that little linear system for the joint step that drives both errors toward zero together, and repeat. The one wrinkle is familiar — a balloon makes a loan's value jump at the balloon date, so when the smooth two-dimensional search stalls the program checks the balloon dates explicitly, the same “step function” care as in one dimension.

10.6 Round-trip proof: the inverse really inverts

Having two engines turns a hope into a check. An inverse solver should undo the forward calculation exactly, and that can be tested directly: compute a payment forward, feed the result back to solve for the original amount or rate, and confirm the original returns. Both engines pass these round-trips and agree with each other — strong evidence that the inversions are genuine inverses, not approximations that merely look right on a few examples. And where an inversion is genuinely *forbidden* — solving the date of a life-contingent payment from its value, where the survival weight depends on the very date sought and the error has no clean single sign-change — both implementations decline rather than return a number a confused search happened to stop at. Recognizing when a problem is ill-posed is as much a part of the mathematics as solving the ones that are.

11 The subtle frontiers

Re-deriving the mathematics a second time, and demanding the two engines agree to the cent, forced into the open a set of edge cases that a single implementation tends to leave implicit. They are worth naming, because they are exactly where naive financial software goes wrong.

Frontier	The mathematical subtlety
Zero interest rate	The annuity factor becomes $0/0$; the correct value is the plain payment count. Both engines detect the neighborhood and substitute the limit.
Growth equals discount	An escalating stream whose growth equals the discount rate has every payment worth the same today; the geometric ratio is 1 and the value is the count times one payment. Again a limit substituted by hand.
Perpetuity	An endless stream has a finite value only if discounting outruns growth; otherwise the honest answer is “infinite,” which the program reports rather than hides.
Odd first period	When the first period is not a whole period, its interest is prorated — and the required payment is adjusted accordingly. The interaction of this stub with payment-in-advance and with balloons is one of the finer points the two-engine effort pinned down precisely.
Payment in advance	An annuity-due discounts every payment one period less; solving for its payment when the payment is the unknown needs the annuity-due form, not the ordinary one — a distinction easy to miss and now made explicit.
Per-payment survival weighting	A life annuity's survival weight differs at every payment date, so its schedule must report a probability <i>per payment</i> , not a single average for the stream. Aligning the modern engine's output to show this per-payment detail — matching the original's tables exactly — was a recent refinement.

None of these are exotic; each is a place where the clean formula has a removable singularity or a hidden case split, and where a defensible-looking shortcut gives a wrong answer. The value of having derived the mathematics twice is that the program's behavior at every one of these frontiers is now explicit and checked, not merely assumed.

12 How we know it is right

Correct-looking formulas are not enough for software that values money. Confidence in Per%Sense rests on several independent supports.

Two independent derivations agree. The strongest evidence is structural: the same financial mathematics was implemented twice, independently, and the two engines produce the same figures to the cent across a very wide range of inputs — ordinary and degenerate, simple and heavily-optioned. Two independent constructions can both be wrong only if they share a mistake, which becomes vanishingly unlikely as the agreement is tested across more cases. This is the same logic by which a result re-derived by a second mathematician gains credibility.

Agreement against known answers. Beyond agreeing with each other, the engines are checked against independently known outputs — figures computed by hand or by standard formulas for textbook cases (a plain annuity, a pure endowment, a term-insurance value, a two-life annuity). Matching an external authority guards against the possibility that both implementations inherited the same misconception.

Internal consistency — round-trips. Every backward solve is required to undo its forward calculation: solve for a payment, then recover the original loan amount from that payment, and the original must return. Passing these round-trips confirms the inverse relationships are genuine inverses, not approximations that merely look right on a few examples.

Deterministic, defined arithmetic. Money is carried to the cent with a fixed tie-breaking rule (ties round toward zero), applied uniformly; and the formulas use accurate series approximations in the neighborhoods where naive computation would lose precision (near zero rates, near unit logarithms). The result is reproducible to the penny and traceable to a defined calculation — the auditability that financial software demands.

The one honest exception. Every support above applies in full to mortgages, loan amortization, and present value. The life-contingency engine (Section 8) is correct against canonical actuarial mathematics and satisfies the recovered usage contract, but because the original mortality core was lost, three details — the table's exact numbers, the age-interpolation convention, and the death-benefit discretization — cannot be checked against the original. Its confidence is therefore high in structure but capped in fidelity, and any answer it produces should be read with that caveat until the original table or a worked original example is recovered.

13 Why it all works

Step back and the whole edifice rests on one idea, extended in layers — and now realized twice over.

At the base is the time value of money: a future amount is worth its discounted value today, discounting being multiplication by an exponential factor. Add that a *regular* stream of discounted payments is a geometric series, and the valuation of any level stream collapses into one annuity formula — the engine. A loan's payment is the annuity inversion; its schedule is a one-line recurrence iterated; a mortgage's relationships are annuity relationships; a present-value worksheet is annuities and lumps summed to a common date; an escalating settlement is an annuity with a modified ratio; a life annuity is the same sum with each term weighted by a survival probability. Nothing genuinely new is introduced as the problems grow richer — only more elaborate cash-flow shapes through the same discount-and-sum machinery. On top sits the inverse problem — solving for any one quantity — handled by clean algebra where the unknown is linear and reliable numerical search where it is not. And wrapping everything is a discipline of precision: deliberate handling of the degenerate cases, a fixed rounding rule, and accurate computation at the numerical edges.

That is why Per%Sense works. It is not a collection of separate formulas for separate screens; it is one principle — discount future money and add it up — expressed with enough care to apply to a level loan, an adjustable mortgage with a balloon, an escalating settlement, or a joint-life pension, and give the right answer for the right reason in every case. And because that one principle was derived and built twice, independently, and the two agree, we have more than a working program: we have a cross-checked account of the mathematics itself — complete and trustworthy where it can be verified, and honestly bounded in the single place where it cannot.